Decomposed Combinatorial Clearing via Fisher Market Budget Allocation

Scalable Prediction Market Clearing through Eisenberg-Gale Decomposition

Draft — February 2026

Summary. Prediction market batch auctions with bundle orders face an exponential state space: N groups of mutually exclusive outcomes produce $\prod K_j$ joint states. The welfare-maximizing clearing problem (an Eisenberg-Gale convex program) is structurally tractable but computationally intractable for large coupled components. We show that the Fisher market structure provides a natural decomposition: independently-solved components are coordinated through MM budget allocation, with the optimality condition being *equal utility* across components. The iterative algorithm — mirror descent on the concave coordination problem — converges at rate $O(1/t)$. We give welfare bounds for approximate decomposition, describe an automatic grouping algorithm based on the coupling graph, and propose a *searcher model* in which external solvers compete to clear coupled components, coordinated by the exchange via budget equalization.

1. Introduction

The companion paper (*Prediction Markets Are Fisher Markets*) establishes that batch auction clearing with budget-constrained market makers is an Eisenberg-Gale convex program. The Fisher market isomorphism holds for *arbitrary joint state spaces* — not just single mutually exclusive groups. This paper addresses the computational consequence: the joint state space is exponentially large, and we need to solve the clearing problem anyway.

The problem. Consider N groups of mutually exclusive outcomes, group j having K_j outcomes. The joint state space is $\mathcal{S} = \prod_j \{1, ..., K_j\}$, with $|\mathcal{S}| = \prod K_j$. A *bundle order* is an order whose payoff depends on the joint state: "buy YES on both A and B" pays \$1 only in the joint state where both resolve YES. Such orders create non-separable demand across groups.

When no orders span multiple groups, the clearing problem decomposes into independent per-group programs — the product-LMSR factorization (Chen and Pennock 2007). Cross-group orders break this, coupling groups *transitively*: if orders link A–B and B–C, clearing requires the full $K_A \times K_B \times K_C$ state space. In practice, a handful of bundle orders can connect all groups into one giant component.

The insight. The Eisenberg-Gale structure provides a decomposition mechanism that the linear (risk-neutral) formulation lacks. The key is *budget splitting*: an MM with budget B_k and orders across multiple components can have its budget allocated across components, with each component solving independently. The optimal allocation satisfies a clean condition — equal utility across components — and the iterative algorithm that finds it is *mirror descent* on the concave coordination problem.

This turns an intractable monolithic optimization into a coordination problem: solve components independently, adjust budget allocations, repeat. The components can be solved in parallel, by different machines, or by different *searchers* (external solvers that compete to provide the best cross-group solutions).

Contributions.

1. The *budget decomposition theorem*: the joint EG program decomposes into per-component programs coordinated by budget allocation, with equal utility at optimality (§2).
2. *Mirror descent convergence*: the iterative budget reallocation algorithm converges at rate $O(1/t)$, with each iteration requiring only independent per-component solves (§3).
3. *Welfare bounds* for approximate decomposition: when cross-group orders are dropped or leg-decomposed, the welfare loss is bounded by the contribution of the dropped correlations (§4).
4. *Automatic grouping*: the coupling graph identifies minimal components; a threshold policy decides which to solve exactly vs. approximately. Column generation exploits demand sparsity to solve large components without full state enumeration (§5).
5. *The searcher model*: external solvers compete to clear coupled components, coordinated by the exchange via budget equalization. The Fisher market structure makes this incentive-compatible (§6).

2. Budget Decomposition

2.1. Setup

We use the notation and results of the companion paper. The risk-averse batch auction clearing program over a joint state space $\mathcal{S}$ is:

$$P^{\mathrm{RA}}: \quad \max_{\boldsymbol{q}\in\mathcal{C}} \quad \sum_k B_k \ln U_k(\boldsymbol{q}) + \sum_{j\notin \mathrm{MM}} w_j q_j - C_b(\boldsymbol{D}(\boldsymbol{q}))$$

where $\boldsymbol{D}(\boldsymbol{q}) \in \mathbb{R}^{|\mathcal{S}|}$ is the net demand vector over joint states, $C_b(\boldsymbol{D}) = b \ln \sum_{s\in\mathcal{S}} \exp(D_s/b)$ is the smoothed minting cost, and $U_k(\boldsymbol{q}) = \sum_{i\in \mathrm{MM}_k} L_i q_i$ is MM k's weighted fill.

Suppose the groups partition into *components* $\mathcal{M} = \{C_1, ..., C_M\}$ (we discuss how to choose this partition in §5). Each component C_m has its own state space $\mathcal{S}_m = \prod_{j\in C_m} \{1, ..., K_j\}$, orders $\mathcal{O}_m$, and minting cost C_b^m.

2.2. The Monolithic vs. Decomposed Program

If an MM k has orders in multiple components, the monolithic program optimizes over all of them jointly. The decomposed approach allocates a *budget share* $B_k^m \geq 0$ to each component, with $\sum_m B_k^m = B_k$, and solves each component independently:

$$P_m(\boldsymbol{B}^m): \quad \max_{\boldsymbol{q}_m\in\mathcal{C}_m} \quad \sum_k B_k^m \ln U_k^m(\boldsymbol{q}_m) + \sum_{j\in\mathcal{O}_m \setminus \mathrm{MM}} w_j q_j - C_b^m(\boldsymbol{D}^m(\boldsymbol{q}_m))$$

The *coordination problem* is to find the budget allocation $\boldsymbol{B} = \{B_k^m\}$ that maximizes total welfare:

$$\max_{\boldsymbol{B}\geq 0} \quad \sum_m W_m^*(\boldsymbol{B}^m) \quad \text{s.t.} \quad \sum_m B_k^m = B_k \quad \forall k$$

where $W_m^*(\boldsymbol{B}^m)$ is the optimal welfare of component m given budgets $\boldsymbol{B}^m$.

Theorem 1 (Budget Decomposition). *When no orders span multiple components, the decomposed program with optimal budget allocation achieves the same welfare as the monolithic program. The optimal allocation satisfies, for each MM k active in components m and m':*

$$\ln U_k^{m*} = \ln U_k^{m'*}$$

Equal utility across components: MM k achieves the same weighted fill in every component where it is active.

Proof. When no orders span components, the minting cost separates: $C_b = \sum_m C_b^m$ (product-LMSR). The monolithic program becomes a joint optimization over fills $\boldsymbol{q}$ and budget allocations $\boldsymbol{B}$:

$$\max_{\boldsymbol{q},\boldsymbol{B}\geq 0} \quad \sum_m \left[\sum_k B_k^m \ln U_k^m(\boldsymbol{q}_m) + \text{retail}_m - C_b^m(\boldsymbol{D}^m) \right] \quad \text{s.t.} \quad \sum_m B_k^m = B_k \quad \forall k$$

This is jointly concave: $B_k^m \ln U_k^m(\boldsymbol{q}_m)$ is concave in $(B_k^m, \boldsymbol{q}_m)$ jointly (it is the perspective of the concave function $\ln U_k^m$, and the perspective of a concave function is concave). The remaining terms are concave in $\boldsymbol{q}_m$ and independent of $\boldsymbol{B}$.

By the envelope theorem, the marginal value of budget in component m is:

$$\frac{dW_m^*}{dB_k^m} = \ln U_k^{m*}$$

where U_k^{m*} is the optimal utility at the current budget. (The indirect effect through the optimal fills vanishes by the optimality of $\boldsymbol{q}_m^*$.) The coordination problem $\max_{\boldsymbol{B}} \sum_m W_m^*(\boldsymbol{B}^m)$ subject to $\sum_m B_k^m = B_k$ has the first-order condition:

$$\frac{dW_m^*}{dB_k^m} = \lambda_k \quad \forall m \text{ where } B_k^m > 0$$

Therefore $\ln U_k^{m*} = \lambda_k$ for all active components: MM k achieves the same utility in every component. Since the decomposed problem is a relaxation of the monolithic (the monolithic is free to choose the same budget split), and the monolithic with separated minting cost factors as the decomposed problem, the welfare is identical. □

Remark. The equal-utility condition determines the budget split implicitly: increasing B_k^m increases U_k^{m*} (the ln singularity guarantees this — more budget means more fills). Components with better opportunities for MM k (higher utility per dollar) receive more budget until utilities equalize.

Remark. When orders *do* span components, the decomposition is approximate: cross-component orders are either dropped, leg-decomposed, or handled by a searcher (§4, §6).

2.3. Why Linear Welfare Cannot Decompose

In the risk-neutral (linear welfare) model, MM k's contribution is $\sum_{i\in \text{MM}_k} w_i q_i$ with a hard budget constraint $\text{cap}_k \leq B_k$. There is no natural way to split a hard budget across components: the constraint $\text{cap}_k^m \leq B_k^m$ with $\sum_m B_k^m = B_k$ introduces a combinatorial allocation problem (which component gets how much budget?) on top of the already-intractable clearing problem. The ln utility replaces this discrete allocation with a smooth, differentiable one. This is the computational payoff of the Fisher market structure.

3. Budget Equalization via Mirror Descent

The equal-utility condition (Theorem 1) suggests a natural iterative algorithm. We need to find the budget split that equalizes $\ln U_k^{m*}$ across components. The coordination problem $\max \sum_m W_m^*(\boldsymbol{B}^m)$ subject to $\sum_m B_k^m = B_k$ is concave, and its gradient is $\partial W_m^*/\partial B_k^m = \ln U_k^{m*}$ (envelope theorem). Mirror descent with KL divergence as Bregman divergence gives:

1. **Initialize.** For each MM k, set $B_k^m = B_k / \left|\{m : \mathrm{MM}_k \cap \mathcal{O}_m \neq \emptyset\}\right|$ (equal split across active components).
2. **Solve.** For each component m in parallel, solve $P_m(\boldsymbol{B}^m)$ to get optimal fills $\boldsymbol{q}_m^*$ and utilities U_k^{m*}.
3. **Update.** For each MM k, reallocate:

$$B_k^m \leftarrow B_k \cdot \frac{B_k^m \cdot U_k^{m*}}{\sum_{m'} B_k^{m'} \cdot U_k^{m'*}}$$

   Multiply each component's allocation by its utility and renormalize.
4. **Repeat** steps 2–3 until convergence.

The update is *multiplicative weights*: each component's budget share grows proportionally to the utility it delivers. At the fixed point, U_k^{m*} must be constant across m (otherwise the highest-utility component would attract more budget), matching Theorem 1.

Theorem 2 (Convergence). *The mirror descent algorithm converges to the optimal budget allocation. For smooth W_m^*, the welfare gap satisfies $W^* - W(\boldsymbol{B}^t) = O(1/t)$.*

Proof. The coordination problem is concave: each $W_m^*(\boldsymbol{B}^m)$ is concave (it is the value function of maximizing a jointly concave objective over a convex set). The gradient $\partial W_m^*/\partial B_k^m = \ln U_k^{m*}$ is computable by solving the per-component program and reading off the equilibrium utilities.

The update exponentiates the gradient and renormalizes — this is mirror descent with the KL divergence as Bregman divergence (equivalently, exponentiated gradient ascent). Since $\exp(\ln U_k^{m*}) = U_k^{m*}$, the update takes the simple multiplicative form above.

For concave maximization of an L-smooth function over the simplex, mirror descent with appropriate step size converges at rate $O(LD^2/t)$ where D is the KL diameter of the feasible region (Beck and Teboulle 2003). In our setting, smoothness follows from the continuous dependence of equilibrium utilities on budgets (the EG program has a unique optimum that varies smoothly with parameters in the interior). □

Remark. Each iteration requires solving M independent convex programs — fully parallelizable. In practice, 3–5 iterations suffice for the budget allocation to stabilize (the per-component programs change slowly as budgets shift).

Remark. Warm-starting: after a budget update, each component's program is a small perturbation of the previous one. Interior-point solvers can warm-start from the previous solution, making subsequent iterations much cheaper than the first.

4. Welfare Bounds for Approximate Decomposition

When bundle orders span multiple components, the decomposition is not exact. The cross-component orders must be handled approximately. We consider two strategies and bound the welfare loss of each.

4.1. Dropping Cross-Component Orders

The simplest strategy: exclude all cross-component orders and solve the remaining (exactly decomposable) system.

Proposition 1 (Drop Bound). *Let $\mathcal{O}_\times$ be the set of cross-component orders. The welfare loss from dropping them satisfies:*

$$W^* - W^*_{\text{drop}} \leq \sum_{i \in \mathcal{O}_\times} w_i \overline{q}_i$$

where w_i is the limit price and $\overline{q}_i$ the maximum fill of order i.

Proof. The dropped system has feasible set $\mathcal{C} \cap \{q_i = 0 : i \in \mathcal{O}_\times\}$, which is a subset of the monolithic feasible set, so $W^*_{\text{drop}} \leq W^*$. Conversely, the monolithic optimum fills each cross-component order i by at most $\overline{q}_i$ at welfare per unit at most w_i (the limit price). The within-component orders' fills in the monolithic optimum are feasible for the dropped system (since they don't couple across components), so the gap is at most the cross orders' contribution. □

This bound is crude — it ignores that cross-component orders compete for minting capacity with within-component orders. In practice, the welfare contribution of cross orders is much smaller than $\sum w_i \overline{q}_i$ because many are only partially filled.

4.2. Leg Decomposition

A tighter approximation: decompose each cross-component order into per-component *legs* using marginal payoffs.

A bundle order i spanning components C_a and C_b has payoff $\varphi_i(s_a, s_b)$ depending on the joint state. Given reference prices $\boldsymbol{p} = (p_a, p_b)$, the *legs* are the marginal payoffs:

$$\varphi_i^a(s_a) = \sum_{s_b} p_b(s_b)\varphi_i(s_a, s_b), \quad \varphi_i^b(s_b) = \sum_{s_a} p_a(s_a)\varphi_i(s_a, s_b)$$

The *separable approximation* is $\hat{\varphi}_i(s_a, s_b) = \varphi_i^a(s_a) + \varphi_i^b(s_b) - \mathbb{E}_{\boldsymbol{p}}[\varphi_i]$, and the *interaction residual* is the non-additive part:

$$\delta_i(s_a, s_b) = \varphi_i(s_a, s_b) - \hat{\varphi}_i(s_a, s_b)$$

The residual has zero marginals ($\mathbb{E}_{s_a}[\delta_i] = \mathbb{E}_{s_b}[\delta_i] = 0$) and vanishes iff the payoff is additively separable. The *interaction magnitude* is $\Delta_i = \max_{s_a, s_b} |\delta_i(s_a, s_b)|$.

Proposition 2 (Leg Decomposition Bound). *Let $\mathcal{O}_\times$ be the cross-component orders and $\hat{\varphi}_i$ the separable approximation of each. The welfare gap satisfies:*

$$|W^* - W^*_{\text{leg}}| \leq \sum_{i \in \mathcal{O}_\times} \overline{q}_i \cdot \Delta_i$$

where $\overline{q}_i$ is the maximum fill and $\Delta_i = \|\varphi_i - \hat{\varphi}_i\|_\infty$ is the interaction magnitude.

Proof. For any feasible fills $\boldsymbol{q}$, the welfare with true payoffs $W(\boldsymbol{q}; \varphi)$ and with separable payoffs $W(\boldsymbol{q}; \hat{\varphi})$ differ only in the minting cost argument:

$$W(\boldsymbol{q}; \varphi) - W(\boldsymbol{q}; \hat{\varphi}) = C_b\big(\hat{\boldsymbol{D}}\big) - C_b(\boldsymbol{D})$$

where $D_s = \sum_i q_i \varphi_i(s)$ and $\hat{D}_s = \sum_i q_i \hat{\varphi}_i(s)$. The smoothed minting cost $C_b(\boldsymbol{D}) = b \ln \sum_s \exp(D_s/b)$ is 1-Lipschitz in ℓ_∞: its gradient $\nabla C_b = \pi$ is a probability distribution ($\|\pi\|_1 = 1$), so $|C_b(\boldsymbol{x}) - C_b(\boldsymbol{y})| \leq \|\boldsymbol{x} - \boldsymbol{y}\|_\infty$. Therefore:

$$|W(\boldsymbol{q}; \varphi) - W(\boldsymbol{q}; \hat{\varphi})| \leq \|\boldsymbol{D} - \hat{\boldsymbol{D}}\|_\infty \leq \sum_{i \in \mathcal{O}_\times} q_i \Delta_i \leq \sum_{i \in \mathcal{O}_\times} \overline{q}_i \Delta_i$$

Since this holds for *all* feasible $\boldsymbol{q}$, it holds at both optima: $W^* = \max_{\boldsymbol{q}} W(\boldsymbol{q};\varphi) \leq \max_{\boldsymbol{q}}[W(\boldsymbol{q};\hat{\varphi}) + \sum \overline{q}_i \Delta_i] = W^*_{\text{leg}} + \sum \overline{q}_i \Delta_i$, and symmetrically. □

Remark. The bound is tight when payoffs are nearly separable ($\Delta_i \approx 0$). For the canonical "buy YES on both A and B" at uniform prices, $\Delta_i = 1/4$ — the interaction is 25% of the payoff range. For a conditional order "buy A if B=YES", $\Delta_i = 1/2$ — leg decomposition loses more because the payoff is strongly non-separable.

Remark. Leg decomposition is iterative: the legs depend on reference prices $\boldsymbol{p}$, which come from equilibrium. In practice, one round (using prices from the within-component solution) captures most of the welfare. The bound above holds for any choice of reference prices and is tightest at prices that minimize the interaction.

4.3. When Decomposition Is Exact

Decomposition incurs zero welfare loss when cross-component orders contribute no *irreducible* correlation:

1. **No cross-component orders.** The product-LMSR factorization applies directly (Theorem 1).
2. **Separable cross-component payoffs.** If every cross-component order i's payoff factors as $\varphi_i(s_a, s_b) = \varphi_i^a(s_a) + \varphi_i^b(s_b)$, leg decomposition is exact (there is no correlation to approximate).
3. **Sparse coupling with tight budgets.** When cross-component orders are few and MM budgets are large relative to their welfare contribution, the cross orders' fills are negligible and the decomposition error vanishes.

5. Automatic Grouping

5.1. The Coupling Graph

Definition 1 (Coupling Graph). *The coupling graph $G = (V, E)$ has groups as vertices. An edge (j, j') exists if any order has non-separable payoffs across groups j and j' (i.e., the order's payoff depends on the joint state of j and j', not just their marginals).*

Connected components of G are the minimal sets of groups that must be solved jointly for exact clearing. The state space of component C_m is $|\mathcal{S}_m| = \prod_{j \in C_m} K_j$.

5.2. Threshold Policy

Given a computational budget (maximum tractable state space size $S_{\max}$):

1. Compute connected components of G.
2. For each component: if $|\mathcal{S}_m| \leq S_{\max}$, solve exactly.
3. If $|\mathcal{S}_m| > S_{\max}$: find a minimum-weight edge cut that partitions the component into sub-components each with $|\mathcal{S}| \leq S_{\max}$. Edge weights are the welfare contribution of the cross-group orders on that edge. Cut edges become leg-decomposed orders.

This minimizes the welfare lost to approximation subject to the computational constraint.

5.3. Treewidth and Structured Coupling

The coupling graph (Definition 1) connects the clearing problem to graphical model inference. The minting cost $C_b(\boldsymbol{D})$ requires summing over joint states $\mathcal{S} = \prod K_j$, but this sum factors according to the coupling graph's structure.

Each cross-component order i introduces a *factor* φ_i over the groups it touches. The minting cost gradient $\partial C_b / \partial D_s$ becomes a marginal computation over a factor graph — exactly the setting of belief propagation and junction tree algorithms.

Key observation. If the coupling graph has treewidth τ, the minting cost and its gradients can be computed in $O\Big(\prod_{j=1}^{\tau+1} K_j\Big)$ time via the junction tree algorithm, avoiding the full $\prod K_j$ enumeration. Since each order touches at most κ groups (in our system, $\kappa \leq 5$), factor size is bounded. The bottleneck is the treewidth τ of the coupling graph, which grows with transitive coupling.

In practice, coupling graphs tend to have moderate treewidth: most markets are coupled to only a few others through bundle orders. When treewidth is low (say $\tau \leq 10$ with binary outcomes), exact clearing via message passing is tractable ($2^{11} \approx 2000$ states per junction tree node). When treewidth is high, the threshold policy (§5.2) falls back to approximate decomposition — or to column generation (§5.4).

5.4. Column Generation for Sparse Demand

When a coupled component is too large to enumerate but demand is sparse (most joint states have zero net demand), *column generation* avoids the full state space by iteratively discovering active states.

The minting cost $C_b(\boldsymbol{D}) = b \ln \sum_{s \in \mathcal{S}} \exp(D_s / b)$ sums over all $|\mathcal{S}|$ joint states, but most contribute negligibly: only states with significant demand D_s matter. Restricting to an active set $\mathcal{S}' \subset \mathcal{S}$ gives a relaxation $C_{b'} \leq C_b$ that overestimates welfare. Column generation closes this gap iteratively:

1. **Initialize.** Set $\mathcal{S}'$ to the joint states appearing in at least one bundle order's payoff (typically $|\mathcal{S}'| \ll |\mathcal{S}|$).
2. **Solve.** Run the EG program with minting cost restricted to $\mathcal{S}'$, producing fills $\boldsymbol{q}^*$ and minting level M^*.
3. **Price.** Find the most violated inactive state: $s^* = \arg\max_{s \notin \mathcal{S}'} D_s(\boldsymbol{q}^*)$. This is the *pricing subproblem*: compute $D_s = \sum_i q_i^* \varphi_i(s)$ and find the maximum over the product space.
4. **Terminate or add.** If $D_{s^*} \leq M^*$ (within tolerance for $b > 0$), stop — the restricted solution is optimal for the full problem. Otherwise, add s^* to $\mathcal{S}'$ and repeat.

The pricing subproblem — $\max_s \sum_i q_i \varphi_i(s)$ — is a sum of low-arity functions over a product space (one factor per order, each touching at most κ groups). This is MAP inference on the same factor graph as §5.3: solvable in $O\Big(\prod K_j^{\tau+1}\Big)$ time when treewidth τ is low, and often tractable in practice even at higher treewidth because single-group orders contribute constants (they do not vary across the product space).

The practical advantage: each iteration solves a *small* EG program (over $|\mathcal{S}'|$ states) rather than the full $|\mathcal{S}|$-state program. When demand is sparse — the typical case, since bundle orders are rare relative to single-group orders — only a handful of joint states ever enter $\mathcal{S}'$.

Remark. Column generation and treewidth (§5.3) are complementary. Treewidth exploits graph structure to compute exact gradients efficiently. Column generation exploits demand sparsity to avoid enumerating inactive states. When treewidth is low, use message passing. When treewidth is high but demand is sparse, use column generation. When both are unfavorable, fall back to approximate decomposition (§4).

6. The Searcher Model

The decomposition framework enables a *market for clearing services*, analogous to Proposer-Builder Separation (PBS) in Ethereum.

6.1. Architecture

1. **The exchange** solves per-group clearing (the easy, separable part) and coordinates budget allocation across components.
2. **Searchers** are external solvers that identify profitable cross-group opportunities. A searcher takes a set of coupled groups, solves the joint EG program over their state space, and submits a *clearing proposal*: fills for the cross-group orders plus the induced prices.
3. **Coordination.** The exchange combines proposals from multiple searchers via budget equalization (Theorem 1). Each searcher's component receives MM budget proportional to the utility it delivers.

6.2. Incentive Structure

A searcher that finds a higher-welfare solution for a coupled component attracts more MM budget (via the budget equalization mechanism), earning more from the bid-ask spread. Searchers compete on *welfare extraction*: the one that best exploits cross-group correlations wins the budget allocation.

This is incentive-compatible: searchers are rewarded exactly for the welfare they create beyond what leg decomposition provides. The exchange does not need to know how to solve the combinatorial problem — it outsources it to competing searchers and coordinates via the Fisher market mechanism.

6.3. Connection to PBS

In Ethereum's PBS, builders compete to construct blocks (bundles of transactions). The proposer selects the most valuable block. In our model:

Component	**PBS (Ethereum)**	**Clearing decomposition**
Builder/Searcher	Constructs blocks	Solves coupled components
Proposer/Exchange	Selects best block	Coordinates via budget equalization
Bid	Block value (MEV)	Component welfare (U_k^m)
Selection	Highest bid wins	Mirror descent (smooth)

The key difference: PBS is winner-take-all (one block selected); our model is *proportional* (multiple searchers' solutions coexist, weighted by welfare). This follows from the ln utility — the smooth allocation again.

7. Discussion

7.1. What We Proved

The Eisenberg-Gale structure turns the intractable combinatorial clearing problem into a tractable coordination problem:

Problem	**Linear welfare**	**Log welfare (EG)**
Single group	LP (easy)	Convex program (easy)
Multi-group, no bundles	Independent LPs	Independent EGs + budget split
Multi-group, bundles	Intractable	Decompose + mirror descent

Budget coordination	Combinatorial	Smooth (equal utility)

7.2. Open Questions

1. **Tight welfare bounds.** Our bounds for approximate decomposition are worst-case. Can we give instance-dependent bounds based on the spectral properties of the coupling graph?
2. **Online / streaming setting.** Orders arrive continuously. Can mirror descent be run incrementally (updating budgets as new orders arrive) rather than from scratch each batch?
3. **Searcher collusion.** If searchers collude, they can extract more MM budget than competitive equilibrium would allocate. What mechanisms prevent this?

This paper is a companion to Prediction Markets Are Fisher Markets *(2026), which establishes the Eisenberg-Gale structure that this paper exploits computationally.*